Practice Question

Create a class **Customer** with the following attributes

- ***customerId*** which is an integer value
- ***customerName*** which is a string value.
- ***customerType*** which could be either REGULAR, PREMIUM or VIP
- ***customerTransactionAmounts*** which is a container that holds the amount values of the last 5 transactions made by the customer on an online store.
- **customerStoreCredits** which is a number indicating the amount of balance or credits a customer has in their account.
- ***customerAccount*** which is a pointer to an instance of Account.
- Ability to use + operator to add the customerStoreCredits of 2 **Customer** instances.
- Ability to gain read-write access to all data members of the class.
- Ability to use **<< *operator*** to display data values of an object of the **Customer** class.
- A parameterized constructor to construct a new **Customer** by passing values for each data member of the class.
- A parameterized constructor that can construct a new **Customer** using just the Id value.

Create a class **Account** with the following attributes

- ***_id*** which is an alphanumeric string.
- ***_balance*** which is a float value.

Create a functionalities.h and functionalities.cpp file with the following functions

- A function to find the ***customerId*** of the **Customer** whose combined ***customerTransactionAmounts*** value is the highest.
- A function to find and return a container of **Customer** objects whose ***customerType*** matches the type passed as the second argument.
- A function to return a container of all **Customer** instances whose ***customerStoreCredits*** are between 100 and 200 (both values included) and _balance in ***customerAccount*** component is over 5000.

- A function to find the **Customer** instance with the lowest and highest ***customerStoreCredits*** and print the combined value for these credits on the standard output.
- A function to find the average ***customerStoreCredits*** of all customers whose customer type matches the type provided as the second argument.
- A function to find if all **Customer** instances in a container are of the ***customerType*** passed as the second argument.
- A function to find and return the count of **Customer** instances whose ***customerType*** is REGULAR and ***customerAccount _balance*** is over 1000

Demonstrate all functionalities by creating an appropriate main function and invoking all relevant functionalities.

Note:

1. The code must utilize modern CPP concepts including CPP 17 wherever applicable.
2. Comments for explanation of complex logic in your code is mandatory